Template-free equality in a seven-rule presentation of iPRA

Research prepared for Vladimir Reshetnikov

November 2026

Abstract

The supplied equality template can be eliminated from the native ConciseIPRA presentation. Replace equality transport by the deterministic rule

$$\frac{s = t \qquad u = v}{v = R_s^t(u)},$$

where R_s^t replaces every exact free occurrence of s by t in a single root-first pass. The rule takes only two premise proofs: no template, path, mask, or selected occurrence is supplied.

The main lemma recovers selective replacement from this global operation. A computation term chosen larger than the entire desired context marks exactly the occurrences to be changed. Computation at zero and the new rule alone then recover all ordinary equational transport. Combined with the repository's existing witness-sensitive introduction, the resulting seven-rule calculus proves exactly the native sequents, in arbitrary contexts. Its full first-order theory is consequently bi-interpretable and equiconsistent with iPRA, with the same numerical domain and actual equality.

This is a precise reduction of the primitive equality interface, not a proof of absolute minimality. Complete checkers, matched comparisons, proof-producing examples, rejection controls and reproducible measurements are included. The tagged checker improves on the comparable one-template implementation. It does not beat the smallest compact programs, whose different input contracts are explicitly distinguished.

Contents

1 The result and its comparison boundary **4**
1.1 The precise simplification 4
1.2 Pinned sources and the moving proposal 4

2 The complete primitive presentation **5**
2.1 Numerical terms, formulas and scopes 5
2.2 The fixed replacement operation 5
2.3 Exactly seven rules 6

3 Recovering all equality reasoning **7**
3.1 The elementary equality operations 7
3.2 A marker that cannot occur accidentally 7
3.3 Selective congruence from global replacement 8
3.4 The converse: G is native equality reasoning 9

4 Recovering the native logical rules **9**
4.1 Structural preliminaries 9
4.2 Recovering unrestricted case introduction 9
4.3 Recovering direct equation progress 10

5 Exact equivalence and both interpretations **10**
5.1 The precise iPRA target 11
5.2 Ordinary logical connectives are definable 11
5.3 The two translations and their composites 12

6 Executable artifacts and size accounting **12**
6.1 One traversal with three precise modes 13
6.2 Whole-program measurements 13
6.3 Executed checks and their limits 14
6.4 A compact forward supplement 14
6.5 Current comparison programs and their input contracts 15

7 Verification status, limits and reproduction **16**

Inherited foundations **18**

A Complementary introductions with one local assumption **18**
A.1 Complete native syntax and rule inventory 18

A.2 Exact equivalence and the unused-witness construction . . . 19
A.3 Induction and numerical cases separate cleanly . . . 20
A.4 The entire ordinary logical interface . . . 21
A.5 Whole-domain realizations and both formula composites . . . 23
A.6 What the theorem does and does not claim . . . 23

B Arithmetic strength and both ordinary interpretation composites 24
B.1 The arithmetic dependency order . . . 24

C Internal scalar predecessor bootstrap 25
C.1 Elementary terms and two finite cycles . . . 26
C.2 A ramp with a global scaling identity . . . 27
C.3 A comparator from scalar iteration . . . 27
C.4 Saturated iteration gives predecessor . . . 28
C.5 Dependency boundary . . . 29

D From literal-equation induction to full intuitionistic PRA 30
D.1 The exact hypotheses . . . 30
D.2 Constructor separation and two elementary tests . . . 31
D.3 Four elementary uses of equation induction . . . 32
D.4 An equation invariant that reflects actual equality . . . 32
D.5 Compilation of all quantifier-free formulas . . . 34
D.6 Recovery of induction and the full theory . . . 35

E Pairing, primitive recursion, and internal numerical composites 35
E.1 Pairing from a bounded square-root search . . . 35
E.2 Compiling each primitive recursion . . . 36
E.3 Why both numerical round trips are internal . . . 37

1 The result and its comparison boundary

The proposal is a calculus $\mathcal{G}$ with the same numerical terms and native formulas as the repository's $\mathcal{N}$ (`NPrf`). Its central property is

$$\Gamma \vdash_{\mathcal{G}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A \tag{1}$$

for every finite context and every native formula, including open sequents. Thus the change does not merely preserve closed equations or consistency strength. It preserves the formula language and all derivability needed for both interpretation composites.

The witness-sensitive introduction is inherited from `NativeOneIntro.lean` [2]. The contribution here is to replace a user-supplied equality context by a fixed global syntax operation, and to prove that no equality strength is lost. The completeness argument uses an oversized computation marker; it does not invoke the logical rules, induction, a numerical case distinction, or substitution into a derivation.

1.1 The precise simplification

Native equality transport supplies an equation template, or equivalently two numerical templates. The newer `one_context` proposal supplies one numerical template. The present rule supplies none:

$$\begin{array}{ll} \text{one-context predecessor} & \text{this proposal} \\ \dfrac{s = t \quad H(s) = v}{v = H(t)} & \dfrac{s = t \quad u = v}{v = R^t_s(u)}. \end{array}$$

The selected occurrences become a derived proof construction, rather than a parameter of an individual equality inference. This is the exact mathematical simplification claimed.

It is not enough to count rule headings: arbitrary machinery could be hidden in a rule. We therefore define R completely, provide an executable checker, and measure whole programs. A smaller primitive certificate interface need not give the smallest implementation or the shortest proofs. In particular, the current compact byte record is not improved by this implementation. No language-independent simplest possible axiomatization is established.

1.2 Pinned sources and the moving proposal

The inherited mathematical sources were copied from ProveIt commit

`a4e5e96c456303cf231eab2aeeb3291d8889833f.`

The source directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. Paths, immutable URLs, Git blob identities and SHA-256 hashes are in `evidence/sources.json`. The copied files and original MIT No Attribution license are included.

The repository advanced during the investigation. The filed comparison was updated through `521884df22e8f2dac0b7066095a42ae975ea4166`, then checked once more at the frozen final boundary

`2a724ea8ec6665f91aa74c892414584b36ca698d.`

The latter check includes the additional `checked_refinement`, `one_sided_transport` and `validated_core` archives.

In particular, `one_context` already contains the one-template crossed rule above. That rule is acknowledged as a predecessor, not claimed as a new result here. The `kernel_refinement` package also contains a supplied single-occurrence context rule [3]. Comparison sources, hashes and the bounded novelty audit are recorded in `evidence/comparison_sources.json` and `evidence/comparison_boundary.json`.

The novelty claim is limited to the inspected proposal and packages. Equality by rewriting is familiar mathematics; no literature-wide priority claim is made.

2 The complete primitive presentation

2.1 Numerical terms, formulas and scopes

There is one numerical sort. Terms and formulas are finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

In $I(n, a; u.h)$, only u is bound, and only in h; the counter and seed are outside its scope. The update may contain fixed numerical parameters. The two variables bound by D are distinct.

The intended ordinary readings are

$$\mathsf{Q}_y B(y) := B(0) \to \forall y\, B(y),$$
$$\mathsf{D}_{x,y} B(x, y) := \exists x (B(x, 0) \to \forall y\, B(x, y)).$$

These are interpretations into ordinary intuitionistic logic, not extra primitive axioms. The designated contradiction is $0 = S0$. All substitutions are capture avoiding; bound variables are identified up to alpha conversion, or represented canonically by de Bruijn indices.

Contexts Γ are finite reusable collections of formulas. Proofs are finite trees. Computation axioms are available in every context.

2.2 The fixed replacement operation

For terms s, t, u in the same numerical context, define $R_s^t(u)$ by a single structural traversal:

1. Test the current subtree for exact equality with the appropriately lifted copy of s. If it matches, return the corresponding lifted copy of t, and stop traversing this subtree.
2. Otherwise preserve the constructor and recurse on its term children. Under an iterator's update binder, lift both s and t by one additional binding level.
3. An unmatched variable or zero is unchanged.

The root test precedes recursion. Inserted targets are never traversed, and newly created matches are never revisited. Thus replacement is total on finite syntax even when t contains s. In particular,

$$R_s^t(s) = t.$$

A bound state variable must not be confused with a free source variable. For example, replacing a free x by 0 under $u.h$ changes lifted occurrences of x and leaves the bound u untouched.

This is deterministic replacement of all exact free occurrences in one pass. It is not repeated rewriting to a normal form. No termination ordering, normalization theorem or unbounded search is part of the rule.

2.3 Exactly seven rules

1. Hypothesis.

$$\frac{A \in \Gamma}{\Gamma \vdash A}. \tag{Hyp}$$

2–3. Computation. For every well-scoped instance,

$$\Gamma \vdash I(0, a; u.h) = a, \tag{I_0}$$
$$\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I_S}$$

4. Global crossed replacement.

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash u = v}{\Gamma \vdash v = R^t_s(u)}. \tag{G}$$

All four endpoints are numerical terms. Sources are unrestricted, including literal zero. There is no supplied term or formula template. Reflexivity, symmetry, transitivity and general congruence are not additional primitive rules.

5. Closure elimination.

$$\frac{\Gamma \vdash \mathsf{Q}_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{QE}$$

The term t is arbitrary, subject to scope and capture avoidance.

6. Paired-binder elimination.

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x, y) \qquad \Gamma, \mathsf{Q}_y B(b, y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

The eigenparameter b is fresh for Γ, A and the free parameters of B. It is local to the second premise.

7. Witness-sensitive introduction. Define a decidable test of finite syntax by

$$\eta(B, t, z) = \begin{cases} z, & B \text{ is a literal equation and } t \text{ is literally } 0, \\ 0, & \text{otherwise.} \end{cases}$$

Then the single introduction rule is

$$\frac{\Gamma, B(t, \eta(B, t, z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}. \tag{DI}$$

Here z is fresh for Γ, t and the free parameters of B. Literal zero means the zero constructor, not a term provably equal to zero. The witness-sensitive test is inherited from the existing proposal.

There are no further logical or arithmetic axioms or inference rules. Successor injectivity, separation, numerical coverage and induction are derived through the equivalence below. Structural rules are metatheorems, not extra proof packets.

3 Recovering all equality reasoning

The following argument uses only I_0 and G. It is valid in every retained context, with arbitrary numerical parameters.

3.1 The elementary equality operations

Lemma 3.1 (Common-left joining)**.** *From $s = t$ and $s = u$, derive $u = t$.*

Proof. Use the two equations as the major and minor premises of G. Its replacement starts with a root match, so $R^t_s(s) = t$. □

Lemma 3.2 (Reflexivity, symmetry and transitivity)**.** *The usual elementary equality operations are derivable.*

Proof. Put $J(a) = I(0, a; w.0)$. The axiom $J(a) = a$, used twice in Lemma 3.1, yields $a = a$.

Given $a = b$, join the major $a = a$ with the minor $a = b$ to obtain $b = a$.

Given $a = b$ and $b = c$, first obtain $b = a$ by symmetry; join the major $b = c$ with the minor $b = a$ to obtain $a = c$. □

No general congruence is assumed in these derivations. Only a root match is used.

3.2 A marker that cannot occur accidentally

Let $H(x)$ be a numerical term template with a distinguished free variable x, distinct from the passive parameters in s, t. There may be any finite number of distinguished occurrences, including zero. Write $|T|$ for the number of constructor and variable nodes of a term, including the update body of every iterator. Renaming and de Bruijn lifting preserve this number.

Choose

$$N = 1 + |H| + |s| + |t|, \qquad K = I(0, s; w.S^N 0). \tag{2}$$

The closed update ignores w. The whole term K need not be closed: it retains the free parameters of s. Its syntax is larger than H, s, t, and I_0 immediately proves $K = s$.

Lemma 3.3 (Exact marking)**.** *The only capture-safe occurrences of K in $H(K)$ are the copies inserted at the distinguished occurrences of x. Consequently,*

$$R^s_K(H(K)) = H(s), \qquad R^t_K(H(K)) = H(t) \tag{3}$$

as finite syntax, up to the agreed binder convention.

Proof. Consider a possible occurrence of a subtree equal to the appropriately lifted K. Tree occurrences are disjoint or nested.

If it is disjoint from every inserted copy of K, it is entirely inside a passive part of H, which has fewer nodes than K. If it is properly inside an inserted copy, it is a proper subtree of that copy and again has fewer nodes. If it properly contains an inserted copy, it has more nodes than K. None of these can be equal to K.

The remaining possibility is an entire inserted copy of K. Under a binder the planted copy is lifted, and the comparison source is lifted by the same amount. Lifting preserves node count, so the argument is unchanged. G stops at each matching root and replaces exactly those copies, which gives both displayed syntactic identities. □

The identities in (3) describe a finite syntax algorithm. They are not object-theoretic uses of the equality rule we are trying to derive.

3.3 Selective congruence from global replacement

Theorem 3.4 (Arbitrary numerical congruence)**.** *For every numerical template H,*

$$\Gamma \vdash s = t \quad \Longrightarrow \quad \Gamma \vdash H(s) = H(t).$$

Proof. Choose K as in (2). The axiom $K = s$ and the premise $s = t$ give $K = t$ by derived transitivity. Derived reflexivity gives $H(K) = H(K)$.

Apply G with major $K = s$ and this reflexive minor. By (3), the result is $H(K) = H(s)$. Apply G with major $K = t$ and the same reflexive minor. The result is $H(K) = H(t)$. Common-left joining, with the latter equation as major, gives $H(s) = H(t)$. □

The compiler uses H to construct ordinary terms and computation certificates. The primitive G packet still contains only two proofs. The checker receives no hidden occurrence set, template or proof of a syntactic side condition. Certificate size may increase.

Theorem 3.5 (Full equational transport)**.** *The following rule is derivable:*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash a(s) = b(s)}{\Gamma \vdash a(t) = b(t)}. \tag{Tr}$$

Proof. Theorem 3.4 gives $a(s) = a(t)$ and $b(s) = b(t)$. Derived symmetry and transitivity compose the chain

$$a(t) = a(s) = b(s) = b(t).$$

□

Why passive occurrences matter. Suppose $s = t$ and $H(x) = I(x, s; w.s)$, where the two displayed passive copies of s are to stay fixed. Directly applying R_s^t to $H(s)$ would also replace them. Instead form $H(K) = I(K, s; w.s)$. Since K is too large to occur in the passive parts, the two global replacements produce exactly

$$I(s, s; w.s), \qquad I(t, s; w.s).$$

The marker proof therefore establishes selective congruence even in the case where naive all-occurrence replacement is insufficient.

3.4 The converse: G is native equality reasoning

Lemma 3.6. *Every G inference is derivable in the native calculus.*

Proof. Mark the occurrences that the fixed root-first algorithm replaces in u. They are disjoint because traversal stops at a match. Abstract just those occurrences by a fresh hole to obtain a numerical template H. Capture-safe matching ensures, as syntax,

$$u = H(s), \qquad R_s^t(u) = H(t).$$

Native equational transport changes $H(s) = v$ to $H(t) = v$. Native symmetry gives $v = H(t)$, exactly G's conclusion. If no occurrence is replaced, H is constant, which is permitted. □

Thus I_0 together with G recovers all the equality machinery that G itself needs for soundness in the old calculus. This equality bootstrap has no dependence on induction or the introduction rules.

4 Recovering the native logical rules

4.1 Structural preliminaries

Lemma 4.1 (Renaming, weakening and cut)**.** *Injective renaming of free parameters, alpha renaming of local parameters, weakening of retained contexts and cut are admissible in $\mathcal{G}$.*

Proof. Injective renaming preserves exact syntax matches and commutes with the capture-safe replacement operation. It also preserves the equation-root and literal-zero tests in DI.

For weakening, induct on a derivation and rename a local eigenparameter away from added assumptions. For cut, rename local eigenparameters away from the finite data of the inserted derivation, then replace uses of the cut assumption by that derivation. The original numerical terms, matrix roots and witness syntax are unchanged. □

Arbitrary numerical substitution is deliberately not asserted by a direct induction here. It can create new exact matches for G, or turn a variable witness into literal zero and change the DI branch. Its admissibility will follow from exact native equivalence.

4.2 Recovering unrestricted case introduction

The old native case rule is

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{DI-case}$$

Lemma 4.2. *Every DI-case instance is derivable in $\mathcal{G}$.*

Proof. Primitive DI already gives the result unless B is an equation and t is literally zero. In that exception put $w = I(0,0;u.0)$. This is syntactically an iterator, and I_0 proves $w = 0$.

Under $B(w,0)$, derived equational transport gives $B(0,0)$. Cut in the supplied case branch to obtain $B(0,Sz)$, and transport along $0 = w$ to obtain $B(w,Sz)$. These transports are equational because the exceptional matrix is an equation.

Apply primitive DI at witness w. Its syntax selects the case branch and concludes the original $\mathsf{D}_{x,y}B(x,y)$. No witness term occurs freely in that conclusion. Choose the local variables fresh. □

This step is not circular: full equational transport was established from I_0 and G alone.

4.3 Recovering direct equation progress

The native progress rule is

$$\frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash \mathsf{Q}_y E(y)}, \qquad E \text{ a literal equation.} \tag{QI-progress}$$

Lemma 4.3. *Every QI-progress instance is derivable in $\mathcal{G}$.*

Proof. Introduce a dummy witness coordinate x absent from E. Primitive DI with witness zero, matrix $B(x, y) = E(y)$, and the supplied progressive premise gives $\mathsf{D}_{x,y}E(y)$. Apply DE. Its local assumption is $\mathsf{Q}_y E(y)$ and is independent of the fresh witness. Hyp therefore closes the branch with that same formula as conclusion, and the eigenvariable condition holds. □

Dummy-coordinate DI-case similarly derives case introduction of Q for arbitrary native matrices. Equation induction is obtained from QI-progress and QE after supplying its base case.

5 Exact equivalence and both interpretations

Theorem 5.1 (Exact sequent equivalence)**.** *For every finite native context Γ and formula A,*

$$\Gamma \vdash_{\mathcal{G}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A.$$

Both transformations of finite proofs are effective. Numerical terms, formulas, contexts and conclusions are unchanged.

Proof. For $\mathcal{N}$ to $\mathcal{G}$, induct on the native proof. Hyp, I_0, I_S, QE and DE are common. Translate native transport by Theorem 3.5, DI-case by Lemma 4.2, and QI-progress by Lemma 4.3.

For $\mathcal{G}$ to $\mathcal{N}$, translate G by Lemma 3.6. A case-mode DI is native DI-case. For a progress-mode DI, the witness is zero and the matrix is an equation. Native QI-progress applied to $B(0, y)$ gives $\mathsf{Q}_y B(0, y)$. Under $B(0, 0)$, QE gives $B(0, Sz)$; native DI-case with witness zero then gives $\mathsf{D}_{x,y}B(x, y)$. The remaining rules are identical.

All constructions retain arbitrary Γ and use finitely many fresh parameters. Syntax tests are decidable metatheoretic operations on constructors; they do not invoke object-theoretic excluded middle. □

Corollary 5.2 (Numerical substitution)**.** *Capture-free substitution of numerical terms into derivations is an admissible metarule of $\mathcal{G}$.*

Proof. Translate to $\mathcal{N}$, apply its substitution theorem, and translate back. □

This is a derived transformation of proofs, not a claim that substituting into one primitive G or DI node always preserves that primitive instance.

5.1 The precise iPRA target

Here iPRA means full first-order intuitionistic logic with a symbol for each fixed primitive-recursive definition, its defining equations, successor injectivity, successor/zero separation, and induction for quantifier-free formulas with numerical parameters and retained assumptions. The target is the `PRAAx` theory in the pinned `PR.lean` [2]. Arbitrary-formula induction is not added.

There is no internal universal primitive-recursive evaluator. Translations recurse externally on each fixed finite description. An iterator update $h(u, \vec{p})$ is elaborated to a function symbol $i_h(n, a, \vec{p})$ satisfying

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

This is primitive recursion whose update ignores the recursion counter.

In the reverse direction, the inherited scalar arithmetic construction first derives predecessor, then actual-equality reflection and quantifier-free induction, and then pairing and the compiler for every fixed primitive recursion. For a target recursion with base g and step h, its state records the counter and current value:

$$\begin{aligned} V(n, \vec{p}) &= I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \\ f^*(n, \vec{p}) &= H(V(n, \vec{p})). \end{aligned}$$

Here π, L, H are the internally proved pairing and projections. Equation induction proves $L(V(n, \vec{p})) = n$ and the target defining equations. The inherited appendices reproduce the dependency chain; pairing is not assumed to bootstrap predecessor.

5.2 Ordinary logical connectives are definable

The inherited formula selector $\mathrm{sel}(v, A, B)$ equals A at zero and B at a successor. The gate $\mathrm{gate}(v, A)$ is trivial at zero and equivalent to A at a successor. Their definitions recurse on the native formula syntax; the appendices supply the construction and proofs. With the parity iterator par, one can define

$$\begin{aligned} \bot &:= 0 = S0, \\ A \to B &:= \mathsf{Q}_y \,\mathrm{sel}(y, A, B), \\ A \wedge B &:= \mathsf{Q}_y \,\mathrm{gate}(y, \mathrm{sel}(\mathrm{par}(y), A, B)), \\ \exists x\, A(x) &:= \mathsf{D}_{x,y} \,\mathrm{gate}(y, A(x)), \\ A \vee B &:= \exists v \ \mathrm{sel}(v, A, B), \\ \forall x\, A(x) &:= A(0) \wedge \mathsf{Q}_y A(y). \end{aligned}$$

All newly bound names are fresh. For the selector and gate construction on a formula containing Q, first use its proved representation by D with an unused witness coordinate; the appended Eq/D construction then applies. These are definitions with derived introduction and elimination rules, not uncounted logical axioms.

In an ordinary first-order realization one must explicitly retain the bridge $(0 = S0) \to \bot$ to genuine falsity. Positive equations alone admit a singleton structure. The appended ordinary-bridge proof includes this sentence and numerical coverage; neither is silently inferred from a purely positive equational theory.

5.3 The two translations and their composites

Let F be the native-to-iPRA translation: compile every iterator description to its primitive-recursive function and read Q, D by the ordinary formulas in Section 2. Let U be the iPRA-to-native translation: compile each primitive-recursive description to a scalar iterator term and each logical connective to its native macro. Both domains are the whole numerical domain, defined by $x = x$, and both equality interpretations are actual equality.

Theorem 5.3 (Bi-interpretability and equiconsistency)**.** *The full theory presented by $\mathcal{G}$ is bi-interpretable and equiconsistent with iPRA. For every native A and ordinary iPRA formula φ,*

$$A \dashv\vdash_{\mathcal{G}} U(F(A)), \qquad \mathsf{iPRA} \vdash \varphi \leftrightarrow F(U(\varphi)).$$

Both numerical composite isomorphisms have graph $x = y$. In particular,

$$\mathcal{G} \vdash 0 = S0 \iff \mathsf{iPRA} \vdash \bot.$$

Proof. The inherited native/iPRA translations preserve all sequents and have these round trips. The formal source statements are `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` in `NativeBiInterpretation.lean`. Their arithmetic and logical constructions are reproduced in the appendices.

For completeness, the arithmetic composite is not justified merely by agreement on standard natural numbers. Equation induction proves uniqueness for the two defining equations of each iterator or primitive recursion, keeping all parameters fixed. External induction on the finite description shows that its double translation satisfies the same defining equations; internal uniqueness proves equality at every input. The identity graph is therefore internally total, bijective and operation preserving. Structural induction on formulas, using the proved meanings of the logical macros, gives both formula composites.

Theorem 5.1 transfers every native sequent in these arguments to $\mathcal{G}$, without changing any term, formula or domain. Hence it transfers both interpretation composites and the contradiction equivalence, not only mutual interpretability. □

6 Executable artifacts and size accounting

The delivered `code/global_rewrite.py` is a complete standalone checker for the seven-rule calculus. Its input is an exact four-key JSON object containing a nonnegative integer `variables`, a list `context` of retained formulas, a formula `goal`, and a primitive `proof` tree. It validates the full tagged Eq/Q/D language and numerical variable scopes. The input contract includes open sequents with the declared numerical parameters, as well as closed theorems. Acceptance requires a normal exit and exactly `ACCEPT` followed by one newline. The program imports only Python's standard JSON and system modules. Neither a producer nor a theorem database is trusted.

Numerical terms are encoded as `['v',i]`, `['z]`, `['s',a]` and `['i',n,a,h]`; the final argument of the iterator binds one variable. Formulas are `['=',s,t]`, `['q',B]` and `['d',B]`. Variables use de Bruijn indices, with index zero the most recent binder. The body of D has its case variable at index zero and its witness variable at index one. The RW instruction contains only its two premise proofs: `['rw',p,q]`. It contains no numerical or formula context to be instantiated.

6.1 One traversal with three precise modes

The helper `walk(a,t,s,k,d)` uses the same structural recursion for lifting, substitution and global replacement. Here d records the number of binders traversed. When a source pattern is supplied, a node matching the source lifted through d binders is replaced by the target lifted through those binders. The traversal stops at that match. An unmatched variable below depth d remains bound; an unmatched index at least d is changed by k. The supported calls are:

Operation	Source pattern	Index offset
Lifting by k	none	k
Substitution, dropping one variable	variable zero	-1
Successor instance under a retained eigenvariable	variable zero	0
Global replacement	the proved source term	0

The matching test precedes descent into children, which enforces the stated top-down replacement convention. In particular, newly inserted material is not visited again. The pattern and image are ordinary checked terms; the internal value `None` denotes the absence of a pattern and is not numerical syntax.

For DI, the matrix is first instantiated at the supplied witness. The progress hypothesis is retained exactly when the matrix is an equation and the witness is the literal zero term; otherwise the retained hypothesis is its zero case. In each branch, the ambient context is lifted when the eigenvariable is introduced. DE similarly lifts the context and the claimed conclusion before checking its continuation. This implements freshness through the scope discipline rather than by a convention imposed on the certificate producer.

6.2 Whole-program measurements

The original X comparator, `reference/one_context_seven.py`, is *our matched seven-rule adaptation* of the incoming `one_context/direct` checker: its two introduction constructors are replaced by the already established witness-sensitive W introduction. It is not represented as an unchanged published 3,181-byte file. The callback and unified-traversal controls retain that X rule and change only the traversal. Relative to the unified X control, the new program changes only the equality-rule branch.

All rows charge an entire standalone module, including syntax validation, scope checks, inference checking, JSON input and the verdict driver. Actual docstrings are removed, and CPython 3.12.14 `ast.unparse` plus one final newline gives the normalized source. Significant-token counts omit comments, encoding and whitespace markers; AST counts include every node of `ast.walk`. Gzip uses level 9, `mtime=0`, and zlib 1.3.2, including the gzip wrapper.

Complete standalone module	Bytes	Tokens	AST	Gzip-9
One-context X: original traversal	3181	1029	1089	976
One-context X: callback traversal	3215	1059	1124	983
One-context X: unified traversal	3206	1044	1096	987
Template-free global replacement	3147	1027	1072	983

The new module is 34 normalized bytes, two significant tokens and 17 AST nodes smaller than the original-traversal X control, a byte reduction of approximately 1.07%. Its gzip size is seven

bytes larger; compression is reported as supplementary information. With exactly the same unified traversal on both sides, removing the supplied context and its checking work saves 59 bytes, 17 tokens and 24 AST nodes.

These are modest matched implementation improvements. The compact arity-code frontier uses a different certificate language and input contract. The table does not establish a new corpus-wide minimum, and does not provide a lower bound on possible checkers. The mathematical contribution is the elimination of the context parameter, together with the large-marker completeness proof; code size is measured separately.

6.3 Executed checks and their limits

The command `python3 tests/verify_strict.py` regenerates all certificates and the measurement table. The authoritative receipts are `evidence/strict_validation.json` and `evidence/strict_metrics.json`; the latter records source hashes, raw and normalized byte counts, and exact runtime versions.

There are 38 named fixtures: 14 valid derivations and 24 expected rejections. Four standalone programs, each run under ordinary Python and `python -O`, give 304 process checks, with zero mismatches. The X controls receive explicit X translations of the R proofs. Their retained contexts, variable counts and claimed conclusions are identical. A separate fixed-seed audit compares 600 liftings, 200 ordinary substitutions and 200 substitutions without dropping the eigenvariable against the original implementation, and checks 200 root replacements: 1,200 further comparisons, all matching.

Positive cases include large-marker congruence with accidental passive occurrences of the source, simultaneous replacement in both equation endpoints, nested iterator binders, a closed congruence theorem, and equation induction that uses its hypothesis. The independent logical naming construction exercises DI, DE and QE as well. Rejection controls cover bound-variable capture, the wrong top-down convention, a case hypothesis substituted for a required progress hypothesis, incorrect computation endpoints, wrong premise kinds, malformed syntax, Boolean and negative indices, and escaping witnesses.

During review, a preliminary callback traversal failed to preserve an unmatched variable leaf: it attempted to descend into the variable's integer payload. The nested-binder fixtures detected the defect. That prototype and its preliminary size are not the delivered result. The unified traversal handles every variable explicitly, and the final receipts refer to the corrected source hashes.

The implementation audit also checks the inference invariants directly: each checked premise has a well-scoped formula; lifting preserves the bound/free distinction; substitution removes exactly its designated variable when requested; and replacement inserts a term only after lifting it through the encountered binders. These are written source arguments supported by executable cases. The finite tests are not a proof-assistant verification of the Python implementation or of bi-interpretability. As for the baseline, the universal proof-system statement abstracts from finite process memory and recursion limits; a resource failure is never acceptance.

6.4 A compact forward supplement

The directory `compact/` supplies a second implementation, using the inherited raw arity representation and a synthesized conclusion. Each non-hypothesis proof is a pair consisting of a major proof and rule data. When the major is $s = t$, its data are only a minor proof $u = v$; the program

returns $v = R_s^t(u)$. No equality template is present in that packet. This supplement is distinct from the strictly parsed tagged interface measured above: administrative aliases retain their inherited meanings, and a strict public-goal grammar validator is not included.

The compact arithmetic packets omit literal-zero initializers. The new congruence compiler therefore uses the uniform aliases

$$C_s = I(2, 1; u.s), \qquad K = I(0, C_s; u.N),$$

where the closed numeral N is larger in syntax-tree size than the desired context. An allowed successor computation gives $C_s = s$, and an allowed base computation gives $K = C_s$. Root transitivity gives $K = s$, including for the literal source $s = 0$. The oversized-alias proof then applies without presupposing missing computation instances or existential naming.

Complete forward module	Bytes	Tokens	AST	Gzip-9
Global replacement	865	397	464	437
Crossed-rule control	830	384	448	414

These are whole-module normalized measurements under CPython 3.12.14 and zlib 1.3.2. Exact delivered sources contain 862 and 827 bytes and compress to 435 and 411 bytes respectively. The gzip files preserve those exact sources; `compact/metrics.json` records both raw and normalized identities. The two forward programs use different packet layouts, so this pair is not a matched ablation of only the equality rule. Neither program improves the 520-byte compact record.

`compact/verify.py` records 25 global-replacement cases in normal and optimized Python, giving 50 process expectations, plus 17 crossed-rule control cases in both modes, giving 34 more. Two deliberate-mutant runs confirm the supplied closed-false-proof attack on the shared-sentinel defect; the corrected program rejects it. All expectations pass. Positive tests include accidental passive occurrences, both dependent equation endpoints, literal-zero sources, two nested iterator binders, and a DE continuation that actually uses its fresh witness before discharging it. Negative tests cover witness escape, future names, and a non-unary DI goal.

The implementation keeps the replacement-source sentinel distinct from the missing-image sentinel, makes the new source argument keyword-only, and restores the old scope floor before checking a DE result. The README documents these guards and the exact input contract. The scripts run from any working directory with the Python standard library alone. This is finite-tested supplementary code with a written mathematical explanation; no proof-assistant verification of its implementation is claimed.

6.5 Current comparison programs and their input contracts

The final comparison boundary is commit `2a724ea8ec6665f91aa74c892414584b36ca698d`, whose recorded commit time is 2026-11-12 01:58:39 UTC. After the earlier filed snapshot, the last check adds the `checked_refinement`, `validated_core` and `one_sided_transport` archives. The first two explicitly refine implementations of existing mathematical theories. The last retains a supplied numerical template in its crossed equality rule. None of these three reports supplies the present global replacement primitive and oversized-marker proof. This bounds the comparison to inspected material.

The current compact coordinate envelope is

256 tokens / 278 AST nodes / 520 bytes.

Three different programs attain these coordinates. In particular, the 520-byte program has 263 tokens and 303 AST nodes; it replaces the earlier 521-byte profile. Its one-byte improvement increases that profile's AST count from 300 to 303. The complete sources and original package claims are copied unchanged under `reference/current_compact/`.

Comparison program	Tokens	AST nodes	Bytes	Gzip-9
Compact byte profile	263	303	520	329
Compact token profile	256	283	594	346
Compact AST profile	260	278	602	344
Native Eq/D, goal checked	358	413	788	460
Guarded Eq/D, goal checked	407	459	921	507
Native Eq/D, fully typed	390	454	878	498
Native Eq/Q/D, fully typed	401	468	897	505

The compact profiles assume a canonical guarded public goal and omit an included public-goal validator. The 788-byte native profile validates the full Eq/D public grammar, with Q available internally, but does not require exact integer/list typing of every certificate field. The fully typed profiles impose that stronger contract; the 897-byte profile also admits every Eq/Q/D public formula. These distinctions prevent a raw byte comparison from being mistaken for a comparison of identical interfaces. The present tagged and compact implementations retain the contracts stated in their own preceding subsections.

Every displayed source was remeasured locally with CPython 3.12.14 using the whole module after `ast.unparse(ast.parse(source))` plus one LF. The token count excludes comments, layout, encoding and end markers; all AST nodes are counted. Gzip uses level 9, timestamp zero and the complete wrapper, and is informational. These measurements reproduce the imported packages' recorded values.

The three new ZIPs were authenticated against the Git blob identities returned by the repository API. Each copied member was then checked against its ZIP bytes. Archive identities, member paths, SHA-256 hashes, individual input contracts and reproduced metrics are recorded in `evidence/comparison_sources.json`; original claims and metric files are retained beside the sources. The imported packages' entire process test suites were not rerun for this size comparison.

The one-template seven-rule matched control used earlier is a derived control, not an unchanged upstream file. Its exact ancestor is the archived `direct/one_context_checker.py`. Replacing its separate case/progress introduction branches by the existing witness-sensitive introduction produces `reference/one_context_seven.py`, with no other change. The exact difference is included as `evidence/one_context_seven_origin.diff`. Further shared-walker variants are identified separately in the matched implementation audit.

7 Verification status, limits and reproduction

The new results are universal written proofs, independently reviewed within this investigation and accompanied by executable proof certificates. They have not been checked in Lean or Rocq. The inherited Lean sources were inspected, but the complete upstream project was not rebuilt here. Existing Lean theorems belong to the inherited bridge; they are not presented as a formalization of G. The copied Lean files are reference excerpts, not a standalone Lake project.

The checkers are standalone and do not import the proof producers or a theorem database. Positive tests exercise genuine primitive proof packets; negative controls exercise binder capture, wrong occurrence semantics, malformed syntax and rule-scope failures. Finite tests do not replace the universal mathematical proof.

Implementation traps were found and fixed during checking. A generic tree walker must preserve an unmatched variable leaf instead of recursing into its integer payload. In the compact shared opener, the missing-image sentinel must differ from the replacement-source sentinel, and the additional source argument must be keyword-only; otherwise existing arity guards can be bypassed. Its DE test must actually use the opened witness to exercise allocation order. The archived implementations and measurements are the corrected versions.

The main result is therefore a concrete seven-rule alternative with a strictly smaller equality-certificate interface, complete derivability equivalence, explicit interpretation composites, and reproducible implementations. The outstanding optimization problem is whether this template elimination can also improve the smallest complete checker under a fixed compact input contract. No absolute optimum or new compact record is claimed.

Run `python3 reproduce.py` from the extracted directory to regenerate measurements and process receipts. Run `python3 build.py` to flatten the attributed appendices and compile the PDF using TeX Live. The archive includes the editable modular source, a standalone `ipra_template_free.tex`, its PDF, code, proof producers, fixtures, comparison sources, source hashes and the verification record.

Inherited foundations: attribution and dependency boundary

The following five sections are reproduced from the pinned `Research/certified_zero/reference/` sources under ProveIt's MIT No Attribution license. Their original wording and historical attributions are retained. References to a new result in these reproduced sections refer to their older packages, not to novelty in the present report.

Some appended proofs use reusable clauses and a native Eq/D presentation. A closure clause $\langle B(0); y.B(y)\rangle$ has the meaning of $\mathsf{Q}_y B(y)$; the ordinary-bridge appendix explicitly proves the correspondence, including the unused-witness representation of Q. These alternate presentations are inherited foundations. The exact primitive language of this proposal is the Eq/Q/D language of Section 2.

The new mathematical construction and its proof are Sections 2–5.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y} A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t,\eta_\varepsilon(z)) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE,B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A,G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0 = 1). \end{aligned} \qquad \text{(Logical macros)}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \qquad \text{(Coverage)}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0,s,t) = C(1,s,t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u),\vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [5] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [4]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [6]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \leq b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \leq n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \leq i, \qquad R_n(i)^2 \leq n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \qquad \text{(Root invariant)}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \leq n < (q(n)+1)^2. \qquad \text{(Root)}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] Vladimir Reshetnikov et al., *Concise presentations of intuitionistic primitive recursive arithmetic*, ProveIt repository, source snapshot `a4e5e96c456303cf231eab2aeeb3291d8889833f`. `https://github.com/VladimirReshetnikov/ProveIt/tree/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.

[2] ProveIt, *Native.lean, NativeOneIntro.lean, NativeBiInterpretation.lean, BiInterpretation.lean, PR.lean*, same snapshot. Exact copied-file identities and URLs appear in `evidence/sources.json`.

[3] ProveIt, *one_context, kernel_refinement, compact_update, uniform_progress*, filed snapshot `521884df22e8f2dac0b7066095a42ae975ea4166`; additional incoming archives checked through `2a724ea8ec6665f91aa74c892414584b36ca698d`. `https://github.com/VladimirReshetnikov/ProveIt/tree/2a724ea8ec6665f91aa74c892414584b36ca698d/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. See the bounded source audit in the archive.

[4] ProveIt, inherited *Equality reflection and quantifier-free induction*, `Research/certified_zero/reference/equality_reflection.tex`, source snapshot. Historical forward-left attribution retained in the appendix.

[5] ProveIt, inherited *Scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, source snapshot.

[6] ProveIt, inherited *Primitive recursion from scalar iteration*, `Research/certified_zero/reference/primitive_recursion.tex`, source snapshot.